

PEM — Permission Escalation Module

Parent Standard: Harness Architecture Standard

Parent Architecture: **Execution Layer — Enforcement Standard**

Category: Governance & Enforcement

Subcategory: Permission Escalation

Type: Execution Containment Module

Version: 1.0

Status: Canonical · Open Module

Effective Date: 9 May 2026

Compatibility: OOF® Methodology OS™ · **Harness Architecture Standard** ·

Execution Layer — Enforcement Standard · RIS · INTEGROS® · OGL™ · ART · OIB™ · EVIP®

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

Permission Escalation Module defines the structural conditions under which a system may request, receive, restrict, deny, or trace elevated permissions, broader execution scope, or higher-impact authority within a **contained execution** environment.

A system satisfies PEM only if:

- escalation conditions are explicitly defined
- higher permission is not granted by default
- escalation triggers remain bounded by governed logic
- approval, denial, restriction, or review conditions are identifiable
- execution does not silently expand authority through runtime convenience, indirect routing, or implicit access
- inheritance

A system that acquires broader power without governed escalation conditions does not satisfy PEM.

Module Function

PEM defines the escalation-control layer of execution containment.

It ensures that **contained execution** does not become uncontrolled execution through gradual, indirect, or opportunistic expansion of permissions.

The module applies wherever autonomous systems may request:

- broader tool access
- stronger execution rights
- access to restricted resources
- higher operational authority

- permission to cross action boundaries
 - release approval for higher-impact execution
-

Minimum Implementation Framework (MIF)

Step 1 — Define Escalation Scope

The organization must define what kinds of permission escalation are possible.

Minimum requirement:

- escalation categories are explicit
 - higher-impact permission classes are distinguishable
 - undefined escalation scope is excluded from valid operation
-

Step 2 — Define Escalation Triggers

The system must define under what conditions escalation may be requested.

Minimum requirement:

- escalation triggers are explicit
 - escalation is not treated as automatic convenience
 - hidden or open-ended escalation conditions are excluded
-

Step 3 — Define Approval and Denial Logic

The system must define how escalation is granted, denied, delayed, or restricted.

Minimum requirement:

- approval path is explicit
 - denial conditions are identifiable
 - unrestricted self-approval is excluded from valid escalation logic unless explicitly governed
-

Step 4 — Define Boundary Retention

The system must define which boundaries remain in force even after escalation.

Minimum requirement:

- escalation does not erase containment
 - elevated permission remains bounded by defined conditions
-

- broader authority does not silently convert into unrestricted execution scope

Step 5 — Preserve Escalation Traceability

The system must preserve visibility of who requested escalation, why it was requested, how it was resolved, and what changed.

Minimum requirement:

- escalation path is traceable
- later review can reconstruct request, approval, denial, and resulting permission state
- escalation does not become opaque after execution proceeds

Step 6 — Restrict Invalid Escalation

The system must not be treated as valid if permission expands without governed request logic, bounded approval conditions, or traceable change of authority.

Minimum requirement:

- invalid escalation conditions are identifiable
- silent permission growth is blocked or invalidated
- execution urgency does not override escalation governance

Use Case 1 — AI Agent Requesting Higher Tool Access

Scenario

An AI agent operating inside a contained environment needs access to a higher-impact tool, broader filesystem scope, or stronger action surface than originally permitted.

Application

PEM is used to ensure:

- escalation scope is defined
- request triggers are governed
- approval or denial conditions are explicit
- elevated access remains bounded and traceable

Result

The organization gains a stronger basis for allowing necessary permission growth without turning **contained execution** into silently expanding authority.

Use Case 2 — Enterprise Runtime with Restricted Operations

Scenario

A system operating inside an enterprise environment reaches a point where a restricted action may be needed, but that action exceeds the default permission level of the running execution context.

Application

PEM is used to ensure:

- escalation is requested under defined conditions
 - broader authority is not granted automatically
 - review and denial remain available
 - elevated execution stays inside containment boundaries
-

Result

The organization gains a structurally governed escalation layer instead of relying on ad hoc permission expansion during runtime.

Canonical Closing Statement

If permission expands without governed escalation, execution containment has not been **achieved.**

Related Documents

[→ Harness Architecture Standard](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>